Project RESILIENCE-ALPHA

Récapitulatif du jour 1 : ce qu'on a construit, et pourquoi

Lundi 19 octobre 2026 - Workshop EPSI Bac+5 - Cyber-résilience et reprise autonome

Objectif	Expliquer simplement tout ce qui a été fait aujourd'hui, pour pouvoir le réexpliquer à l'oral (jury, entretien, alternance) : les outils et leur rôle, l'authentification, les intervalles de 5 secondes, les problèmes rencontrés et leurs solutions.

Sommaire

- 1. Le projet en 5 lignes
- 2. Ce qu'on a construit aujourd'hui (vue d'ensemble)
- 3. Les outils et leurs rôles
- 4. L'authentification en détail (mot de passe, jeton de 15 minutes)
- 5. Le « truc de 5 secondes » : la surveillance
- 6. Les problèmes rencontrés et ce qu'on en a appris
- 7. Les choix de sécurité faits aujourd'hui
- 8. Les fichiers du projet et les commandes utiles
- 9. Où on en est, et la suite
- 10. Questions possibles du jury
- 11. Mini-glossaire

1. Le projet en 5 lignes

Le sujet demande de construire une **fausse plateforme de paiement**, de l'**attaquer nous-mêmes** depuis un second ordinateur, et de montrer qu'elle **se répare toute seule**, sans qu'un humain intervienne.

Analogie : c'est le groupe électrogène d'un hôpital. Quand le courant est coupé, il démarre sans technicien. Chez nous : l'attaque coupe le service, le système détecte, répare, puis prouve que ça marche à nouveau.

Moment	Ce qui se passe	Qui s'en charge
1. Avant	La plateforme fonctionne, une transaction passe	Notre appli
2. Attaque	Flood de requêtes + brute force sur le login	PC Attaquant
3. Signal	Les chiffres et les logs changent brutalement	Prometheus + Loki
4. Détection	Un modèle repère l'anomalie	Moteur ML (à construire)
5. Réparation	L'IP attaquante est bloquée, le service relancé	Orchestrateur (à construire)
6. Preuve	Une nouvelle transaction réussit	Vérification automatique

> **Règle d'or** Si un humain doit intervenir pendant la démonstration, la note de « remédiation » est de 0/5. Un conteneur qui redémarre ne prouve rien : il faut une vraie transaction réussie après l'incident.

Matériel : le PC Hôte (ton PC) fait tourner toute la plateforme. Le PC Attaquant (PC d'un camarade) lance les attaques. Un troisième PC héberge l'IA (Ollama). Ils sont reliés par le point d'accès Wi-Fi Windows `RESILIENCE-G<n>` (réseau 192.168.137.0/24).

2. Ce qu'on a construit aujourd'hui

Aujourd'hui, on a posé les fondations : la plateforme elle-même, sa porte d'entrée sécurisée, l'authentification, et les « yeux » (surveillance) qui serviront plus tard à détecter l'attaque.



Tout tourne dans des **conteneurs Docker** lancés par un seul fichier, `docker-compose.yml`. Les conteneurs sont rangés dans deux réseaux privés :

Réseau	Contient	Pourquoi
app (172.28.0.0/24)	WAF, API, auth, base, Prometheus	La plateforme. Seul le WAF est joignable de l'extérieur.
monitoring	Prometheus, Loki, Alloy, Grafana	La surveillance, séparée de la plateforme : Grafana ne peut pas toucher à la base.

Les ports ouverts sur ton PC sont tous limités à `127.0.0.1` (ton PC uniquement) : 8081 (WAF), 3000 (Grafana) et 9090 (Prometheus). Rien n'est encore visible depuis le Wi-Fi du groupe.

3. Les outils et leurs rôles

Chaque outil a un rôle précis. Voici une fiche simple pour chacun, avec une analogie.

Outil	Son rôle en une phrase	Analogie
Docker	Fait tourner chaque programme dans une « boîte » isolée (conteneur), identique sur toutes les machines.	Un conteneur maritime : le contenu est le même partout.
Docker Compose	Lance et relie tous les conteneurs d'un coup grâce à un fichier (docker-compose.yml).	Le chef d'orchestre qui démarre tous les musiciens.
PostgreSQL	La base de données : stocke les comptes et les transactions fictives.	Le classeur de la banque.
FastAPI (API métier)	Le programme Python qui reçoit les demandes (login, transactions, état de santé).	Le guichet de la banque.
Service auth	Vérifie le mot de passe et délivre un jeton (JWT) valable 15 minutes.	L'accueil qui contrôle ta pièce d'identité et te donne un badge visiteur.
WAF (Nginx + ModSecurity + règles OWASP CRS)	Lit le contenu de chaque requête web et refuse celles qui ressemblent à une attaque.	Le vigile qui fouille les sacs, pas seulement celui qui regarde le badge.
Prometheus	Relève des chiffres (métriques) toutes les 5 secondes et les garde dans le temps.	Le tableau de bord de la voiture : vitesse, température.
Loki	Stocke les logs (les messages d'événements) avec des étiquettes (service=auth).	Le carnet de bord du conducteur.
Alloy	Lit les logs de tous les conteneurs et les envoie à Loki.	Le facteur qui collecte le courrier.
Grafana	L'écran : affiche chiffres (Prometheus) et logs (Loki).	La loupe et le tableau de bord réunis.

Logs et métriques : deux visions complémentaires

	Métriques (Prometheus)	Logs (Loki)
C'est	Des chiffres dans le temps (ex. 300 requêtes par seconde)	Des phrases d'événements (ex. « login raté pour alice »)
Sert à	Voir **qu'il se passe quelque chose**	Comprendre **quoi** exactement
Exemple chez nous	http_requests_total, login_attempts_total	{"event": "login_failed", ...}

Pourquoi pas Elastic Stack ? Le sujet précise qu'il n'est pas attendu : trop gourmand en mémoire pour le PC Hôte. Loki + Grafana jouent le rôle de « SIEM / fonction SOC » avec bien moins de RAM.

4. L'authentification en détail

L'authentification répond à la question : « qui es-tu ? ». Notre service **auth** fait deux choses : vérifier un mot de passe, puis donner un **jeton** (JWT) qui prouve l'identité pendant 15 minutes.

4.1 Le parcours complet

```
1. Client --> WAF --> API /login --> service auth    "alice + mot de passe ?"
2. auth compare avec l'empreinte (hash) stockée en base
3. Si c'est bon : auth renvoie un jeton signé, valable 15 minutes
4. Client --> WAF --> API /transactions   avec l'en-tête  Authorization: Bearer <jeton>
5. L'API vérifie la signature et la date du jeton, puis traite la demande
```

Pourquoi l'API relaie le login à auth ? Le WAF n'a qu'un seul « backend ». On a donc gardé l'API comme unique porte, qui parle à auth en interne.

4.2 Le mot de passe n'est jamais stocké : on stocke un hash bcrypt

Analogie : un mixeur à smoothie. Tu mets des fruits (le mot de passe), tu obtiens un smoothie (le hash). On peut vérifier qu'un smoothie vient bien de ces fruits, mais on ne peut pas retrouver les fruits à partir du smoothie. Même si la base est volée, les mots de passe ne le sont pas. Bcrypt est volontairement lent, ce qui ralentit aussi les attaques par essais répétés.

```
ok = row is not None and bcrypt.checkpw(creds.password.encode(), row[0].encode())
if not ok:
    event("login_failed", username=creds.username, client_ip=x_client_ip)
    raise HTTPException(status_code=401, detail="identifiants invalides")
```

- Le message d'erreur est **identique** que le compte existe ou non : on n'apprend rien à l'attaquant.
- Chaque échec écrit une ligne JSON `login_failed` dans les logs et incrémente un compteur Prometheus : c'est le signal du brute force.

4.3 Le jeton JWT (le « badge visiteur »)

Un JWT a trois parties séparées par des points : `en-tête.contenu.signature`. Celui qu'on a obtenu commençait par `eyJ...` (c'est normal, c'est du texte encodé).

Partie	Contient	À retenir
En-tête	L'algorithme de signature (HS256)	-
Contenu (payload)	sub = alice (l'utilisateur), exp = date d'expiration (15 min après la création)	Lisible par tout le monde : on n'y met jamais de secret.
Signature	Calculée avec le secret JWT_SECRET, connu seulement de auth et de l'API	Si quelqu'un modifie le contenu, la signature ne correspond plus : le jeton est refusé.

Pourquoi 15 min Un jeton court limite les dégâts s'il est volé : au bout de 15 minutes il ne marche plus, et il faut se reconnecter. C'est aussi pour ça que, pendant nos tests, on devait refaire le login.

4.4 Les réponses possibles

Requête	Réponse	Signification
Login avec le bon mot de passe	200 + access_token	Tu as ton jeton

Requête	Réponse	Signification
Login avec un mauvais mot de passe	401 identifiants invalides	Refusé, et loggé comme login_failed
/transactions sans jeton	401 jeton manquant	L'API exige un badge
/transactions avec un jeton expiré ou falsifié	401 jeton invalide ou expiré	Signature fausse ou date dépassée
/transactions avec un bon jeton	201 + id de la transaction	Transaction fictive enregistrée
/health	200 {status: ok, db: ok}	Public : sert à la supervision et à la preuve de reprise

5. Le « truc de 5 secondes » : la surveillance

Dans la configuration de Prometheus, on a écrit :

```
global:
  scrape_interval: 5s      # une mesure toutes les 5 secondes
  evaluation_interval: 5s
```

Scrape veut dire « aller relever ». Toutes les 5 secondes, Prometheus va lire les compteurs de l'API et de auth (sur le port interne 9100) et les enregistre avec l'heure. Analogie : un relevé de compteur d'eau toutes les 5 secondes au lieu d'une fois par mois : on voit tout de suite une fuite.

5.1 Comment ça marche

```
API  (compteur http_requests_total)   --+
AUTH (compteur login_attempts_total)  --+-- relevé toutes les 5 s --> Prometheus --> Grafana
```

- Un **compteur** (Counter) ne fait que monter : 1, 2, 3... Ce qui nous intéresse, c'est la **vitesse** à laquelle il monte, calculée avec `rate()` (requêtes par seconde).
- En temps normal : quelques requêtes par seconde. Pendant un flood : des centaines. Pendant un brute force : `login_attempts_total{result="failed"}` explose.
- Chaque requête est comptée par méthode, route et code de réponse (200, 401, 403...).

5.2 Pourquoi 5 secondes ?

Intervalle	Avantage	Inconvénient
Court (5 s) - notre choix	On voit une attaque en quelques secondes : indispensable pour une réparation automatique rapide	Un peu plus de travail et de stockage pour Prometheus
Long (1 min)	Très léger	On détecterait l'attaque trop tard, la démo durerait trop longtemps

> **Autre 5 s** Dans le compose, la base de données a aussi une vérification de santé toutes les 5 secondes (`pg_isready`). L'API et l'auth attendent que la base réponde avant de démarrer (`depends_on ... service_healthy`).

5.3 Deux détails de sécurité dans les métriques

- Les métriques sortent sur un **port séparé (9100)** que le WAF n'expose jamais : le public ne peut pas les lire.
- On enregistre le **modèle de la route** (« /transactions »), pas l'URL brute. Sans ça, un attaquant pourrait inventer des milliers d'URL et faire gonfler la mémoire de Prometheus (explosion de cardinalité).

5.4 Les logs : le trajet d'une ligne

```
auth écrit  {"service":"auth","event":"login_failed",...}
   -> Docker garde la ligne -> Alloy la lit
   -> Loki la range (service="auth") -> Grafana l'affiche
```

Dans Grafana, onglet **Explore**, source **Loki**, mode **Code**, la requête `{service="auth"}` affiche ces lignes. Côté chiffres, la source **Prometheus** avec `http_requests_total`.

6. Les problèmes rencontrés et ce qu'on en a appris

Ces problèmes sont de la matière précieuse pour le dossier et l'oral : ils montrent qu'on comprend le système.

Problème	Cause	Solution
Le WAF répondait avec un message « Airflow »	Le port 8080 était déjà occupé par un autre projet (Apache Airflow) sur le PC.	Le WAF écoute sur 8081 côté PC (mapping 127.0.0.1:8081:8080). Un port = une seule adresse par programme.
404 sur /transactions après avoir modifié le projet	Nginx (WAF) traduit le nom « api » en adresse IP une seule fois, au démarrage. L'API a été recréée avec une nouvelle IP, et son ancienne IP a été prise par le service auth : le WAF parlait au mauvais service.	Redémarrage du WAF, puis correctif durable : **adresses IP fixes** pour chaque service (172.28.0.10 à .13).
Loki absent de la liste dans Grafana	Grafana ne relit sa configuration (sources de données) qu'au démarrage.	docker compose restart grafana.
Erreur « parse error » dans Loki	La requête {service="auth"} avait été tapée dans la case « Line contains » du mode Builder.	La saisir en mode Code, ou choisir label=service, valeur=auth.

> **Important** Le problème d'IP est **crucial pour la suite** : la remédiation va redémarrer ou recréer l'API. Si l'API changeait d'adresse, le WAF ne la retrouverait pas et la plateforme resterait en panne même après la « réparation ». Avec les IP fixes, on a testé `docker compose up -d --force-recreate api` : l'API est recréée et la plateforme répond de nouveau, sans toucher au WAF.

7. Les choix de sécurité faits aujourd'hui

À reprendre dans le dossier d'ingénierie (partie « durcissement »).

Choix	Ce qu'il protège
Secrets dans .env, ignoré par Git (.gitignore), modèle .env.example sans valeur	Aucun secret en clair dans l'archive finale (exigence du sujet). Plus tard : Vault ou SOPS.
Conteneurs API et auth sans droits administrateur (utilisateur appuser)	Si l'appli est compromise, l'attaquant a moins de droits dans le conteneur.
Requêtes SQL paramétrées (VALUES (%s, %s, %s))	Protège contre l'injection SQL côté appli.
Validation des entrées (longueurs, montant entre 0 et 10 000)	Réduit les données absurdes ou malveillantes.
Mots de passe hachés avec bcrypt	La base volée ne livre pas les mots de passe.
Jeton JWT de 15 minutes, signé	Limite l'impact d'un vol de jeton.
Même message d'erreur pour mauvais compte et mauvais mot de passe	On ne révèle pas quels comptes existent.
Une seule porte d'entrée : le WAF ; l'API et la base ne sont pas exposées	Réduit la surface d'attaque.
Ports publiés seulement sur 127.0.0.1	Rien n'est visible depuis le réseau du campus pour l'instant.
Métriques sur un port interne séparé, séparation des réseaux app / monitoring	Moins d'accès croisés entre composants.

Risques identifiés (à documenter honnêtement)

Risque	Détail	Piste
Socket Docker monté dans Alloy	Même en lecture seule, il donne accès à l'API Docker : un Alloy compromis pourrait contrôler les conteneurs.	Documenter le compromis ; limiter les droits de l'orchestrateur (même risque).
En-têtes CORS trop permissifs du WAF	Access-Control-Allow-Origin: * autorise n'importe quel site à appeler l'API depuis un navigateur.	Restreindre à l'origine du frontend quand il existera.
Adresse IP client à confirmer	Le champ client_ip des logs de login n'a pas encore été vérifié : la remédiation doit connaître l'IP de l'attaquant (et non celle du WAF).	À contrôler dans Loki demain, et corriger si besoin.

8. Les fichiers du projet et les commandes utiles

```
resilience-alpha/
  docker-compose.yml            <- lance les 8 conteneurs, réseaux, IP fixes
  .env                          <- secrets (jamais dans Git)
  .env.example                  <- modèle sans secret
  .gitignore
  services/
    db/init.sql                 <- crée la table des transactions
    api/   Dockerfile, requirements.txt, app/main.py   <- l'API
    auth/  Dockerfile, requirements.txt, app/main.py   <- l'authentification
    prometheus/prometheus.yml   <- cibles et intervalle de 5 s
    alloy/config.alloy          <- collecte des logs Docker -> Loki
    grafana/provisioning/datasources/datasources.yml   <- Prometheus + Loki
```

Les commandes (à lancer dans PowerShell, dans le dossier resilience-alpha)

Commande	Ce qu'elle fait	Résultat attendu
docker compose up -d	Démarre tous les services en arrière-plan	« Started » pour chaque service
docker compose up --build -d	Idem, en reconstruisant les images après un changement de code	Build puis « Started »
docker compose ps	Liste les services et leur état	Tous « Up » (db et waf « healthy »)
docker compose logs auth --tail 5	Affiche les 5 dernières lignes de logs d'un service	Lignes JSON login_ok / login_failed
docker compose restart waf	Redémarre un seul service	« Restarted »
docker compose up -d --force-recreate api	Recrée l'API de force (test de reprise)	API recréée, WAF intact
docker compose down	Arrête et supprime conteneurs et réseau (les données restent)	« Removed »

Les adresses utiles (dans le navigateur de ton PC)

Adresse	Ce que c'est
http://127.0.0.1:8081/health	L'API, via le WAF : {status: ok, db: ok}
http://127.0.0.1:3000	Grafana (utilisateur admin, mot de passe dans .env)
http://127.0.0.1:9090/targets	Prometheus : les cibles api, auth, prometheus doivent être UP

Conseil RAM : ton PC a 15,8 Go. Arrête Airflow (l'autre projet) pendant le workshop : `docker compose stop` dans son dossier.

9. Où on en est, et la suite

État	Élément
FAIT	Compréhension du sujet, choix de l'architecture et du scénario d'attaque (flood + brute force)
FAIT	Base PostgreSQL + API (login, transactions, health)
FAIT	WAF devant l'API (testé : une attaque XSS reçoit 403)
FAIT	Service d'authentification (bcrypt, JWT 15 min, logs login_failed)
FAIT	IP fixes + test de reprise de l'API (--force-recreate)
FAIT	Prometheus (relevé toutes les 5 s) + Grafana + Loki + Alloy
À FAIRE	Ouvrir le WAF au Wi-Fi du groupe et tester depuis le PC Attaquant (en attente du PC)
À FAIRE	Matrice de risques EBIOS RM, à faire valider par les coachs
À VÉRIFIER	Le champ client_ip dans les logs de login

Demain (mardi) : production

- Frontend React + Tailwind (interface de paiement)
- Vault ou SOPS pour les secrets
- Script d'attaque (flood + brute force) sur le PC Attaquant
- Playbook Ansible
- Modèle de machine learning pour détecter l'anomalie
- Assistant Ollama + RAG sur le PC dédié
- Dossier de gouvernance NIS2 / DORA, PCA / PRA

Mercredi et jeudi

- Orchestrateur de remédiation (blocage d'IP + vérification par une nouvelle transaction) : c'est le critère noté 5/5
- Dashboard pour le jury, chaos engineering, Clip Chrono de 60 secondes sans coupe
- Dossier PDF (8 à 12 pages), présentation .pptx, archive Code.zip sans secret
- **Jeudi : gel du code à 17h, remise des fichiers avant 23h59.** Vendredi : démonstration devant le jury.

10. Questions possibles du jury

Question	Réponse courte
Pourquoi un WAF devant l'API ?	Il lit le contenu des requêtes et bloque les attaques connues (XSS, injection SQL) avant qu'elles atteignent l'appli. C'est notre seule porte d'entrée.
Comment stockez-vous les mots de passe ?	Jamais en clair : un hash bcrypt, à sens unique et volontairement lent.
Qu'est-ce qu'un JWT et pourquoi 15 minutes ?	Un badge signé contenant l'utilisateur et une date d'expiration. Court pour limiter l'impact d'un vol.
Quelle différence entre métriques et logs ?	Les métriques (Prometheus) sont des chiffres dans le temps : elles montrent qu'il se passe quelque chose. Les logs (Loki) disent quoi.
Pourquoi relever toutes les 5 secondes ?	Pour voir une attaque en quelques secondes et déclencher une réparation rapide pendant la démo.
Quel problème avez-vous rencontré ?	Le WAF gardait en mémoire l'ancienne IP de l'API. Solution : IP fixes pour que la remédiation puisse recréer l'API sans casser le WAF.
Quels sont les risques de votre architecture ?	Le socket Docker monté dans Alloy (et plus tard dans l'orchestrateur) donne un accès très puissant ; CORS permissif du WAF à restreindre.
Pourquoi pas Elastic Stack ?	Trop lourd pour la RAM du PC Hôte ; Loki + Grafana couvrent le besoin.

11. Mini-glossaire

Mot	En simple
Conteneur	Une boîte isolée contenant un programme et tout ce qu'il lui faut pour tourner.
API	La porte d'entrée d'un programme : on lui envoie des demandes, il répond.
WAF	Pare-feu applicatif : filtre le contenu des requêtes web.
Hash	Empreinte à sens unique d'un mot de passe.
JWT	Jeton signé qui prouve l'identité pendant une durée limitée.
Métrique / Log	Un chiffre dans le temps / un message d'événement.
Remédiation	La réparation automatique après détection d'un incident.
Brute force	Essayer des milliers de mots de passe jusqu'à trouver le bon.
Flood	Envoyer un flot de requêtes pour saturer un service.
EBIOS RM	Méthode française d'analyse de risques.
NIS2 / DORA	Réglementations européennes de cybersécurité et de résilience.
PCA / PRA	Plan de continuité / plan de reprise d'activité.